Google Developer Groups
On Campus • Zeal College of Engineering & Research

Docker Fundamentals

From "It works on my machine" to deploying anywhere.

By: Rhiya Buranpur

Agenda

- What is Docker and why does it exist?
- Core Concepts - Image, Container, Layer
- Hands-on 1 - pull and run Nginx
- Writing a Docker FIle
- Hands-on 2 - Build your own Image
- Volumes, env, vars & Docker Compose
- Wrap-up and Repo tour

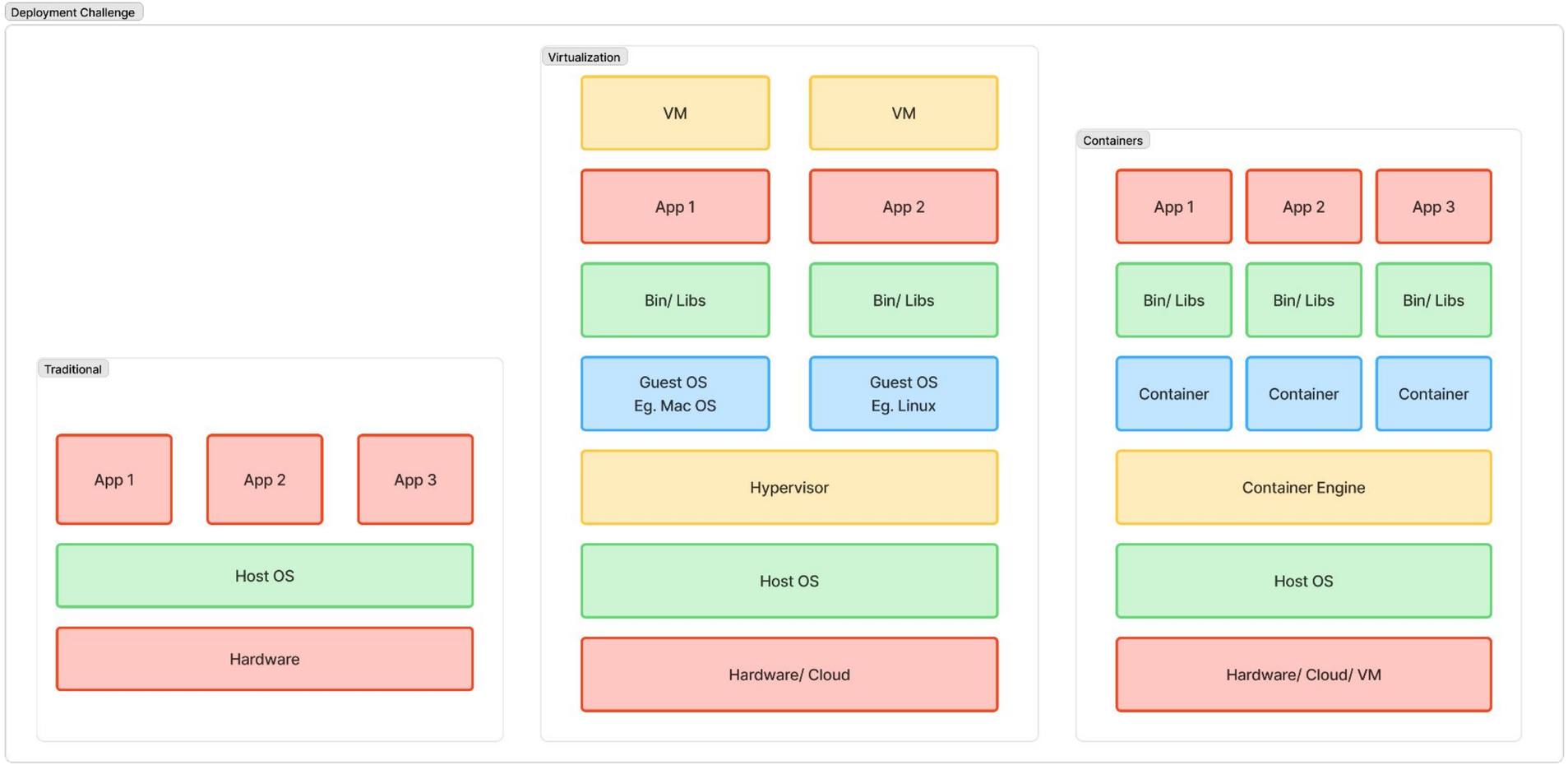
I get that it works on your computer
But we are not going to give your computer to the client

“It works on my machine”

Dev Laptop	Teammate’s Laptop	Production Server
Python 3.9	Python 3.11	Python 3.8
Ubuntu 20	MacOS 14	CentOS 7
Lib v1.2	Lib v1.4	Lib v1.1

Docker solves this by packaging your app and its environment together.

Virtual Machines V.S. Containerisation



Images, Containers & Layers

Image

Read-only blueprint. Like a class in programming. Built from a Dockerfile. Stored in a registry (Docker Hub).

Container

A running instance of an Image. Like an object from a class. Isolated. Ephemeral by default.

Layer

Each Dockerfile instruction creates a layer. Layers are cached and shared between images - keeps things fast and small.

Anatomy of a Dockerfile

```
FROM python:3.12-slim                    --- Base image from Docker Hub
WORKDIR
 /app                                    --- Sets working directory inside container
COPY
 requirements.txt .                      --- Copy requirements first (layer cache!)
RUN
 pip install -r requirements.txt         --- Install dependencies
COPY
 app.py .                                --- Copy your application code
EXPOSE
 5000                                    --- Document the port (doesn't publish it)
CMD
 ["python", "app.py"]                    --- Default command to run on start
```

Port Mapping and Volumes

Port Mapping -p 8080:80

Your laptop:8080 → Container:80

Traffic on host port 8080 is forwarded to port 80 inside the container.

Volumes -v /host/path:/container/path

Mount a host directory into the container. Data persists after the container stops.

-v ./data:/app/data

Environment Variables -e KEY=VALUE

docker run -e APP_ENV=production -e SECRET_KEY=abc123 myapp

Inject config without rebuilding the image — environment stays outside the code.

Multi-container apps in one file

```yaml
services:
  app:
    build: ../app
    ports:
      - "5000:5000"
    depends_on:
      - redis

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
```

- Define all services in one YAML file
- docker compose up -d -> starts everything
- Services talk to each other by name (e.g. ‘redis’)
- depends_on controls start order
- docker compose down -> stops & cleans up

Quick Reference

Image

`docker pull <img>`
Download from DockerHub

`docker images`
List local Images

`docker build -t <n> .`
Build from Dockerfile

`docker rmi <img>`
Remove image

Container

`docker run -d -p 8080:80 <img>`
Run detached + port

`docker ps`
List running containers

`docker stop <n>`
Stop a container

`docker rm <n>`
Remove a container

`docker logs <n>`
View logs

`docker exec -it <n> bash`
Shell into container

Compose

`docker compose up -d`
Start all services

`docker compose ps`
List services

`docker compose logs`
View all logs

`docker compose down`
Stop & remove

Google Developer Groups
On Campus • Zeal College of Engineering & Research

Resources - Github and Docs

Google Developer Groups
On Campus • Zeal College of Engineering & Research

Thankyou For Showing Up!

Let’s Connect!